

All tree traversals begin by passing the root node to the recursive function. The function makes a recursive call and passes each of the root node's children as the argument. Since these child nodes have children of their own, the traversal continues until a leaf node with no children is reached. At this point, the function call simply returns.

The recursive case occurs if the node has any child nodes ❶, in which case a recursive call is made with each of the children as the node argument. Whether or not the node has children, the base case always happens at the end of the function when it returns ❷.

Postorder Tree Traversal

Postorder tree traversal traverses a node's child nodes before accessing the node's data. For example, this traversal is used when deleting a tree and ensuring that no child nodes are “orphaned” by deleting their parent nodes first, leaving the child node inaccessible to the root node. The code in the following *postorderTraversal.py* program is similar to the preorder traversal code in the previous section, except the recursive function call comes before the `print()` call:

Python

```
root = {'data': 'A', 'children': [{'data': 'B', 'children':
[{'data': 'D', 'children': []}]}, {'data': 'C', 'children':
[{'data': 'E', 'children': [{'data': 'G', 'children': []},
{'data': 'H', 'children': []}]}]}, {'data': 'F', 'children': []}]}}

def postorderTraverse(node):
    for child in node['children']:
        # RECURSIVE CASE
        postorderTraverse(child) # Traverse child nodes.
    print(node['data'], end=' ') # Access this node's data.
    # BASE CASE
    return

postorderTraverse(root)
```

The *postorderTraversal.html* program has the equivalent JavaScript code:

JavaScript